



厦门大学
XIAMEN UNIVERSITY

Python 程序控制



1.程序控制



1.1 程序控制 程序控制

- 程序流程图用一系列图形、流程线和文字说明描述程序的基本操作和控制流程，它是程序分析和过程描述的最基本方式

流程图
的七种
基本元
素



起止框



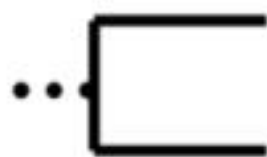
判断框



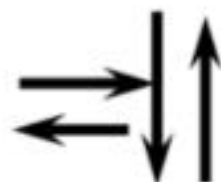
处理框



输入/输出框



注释框



流向线

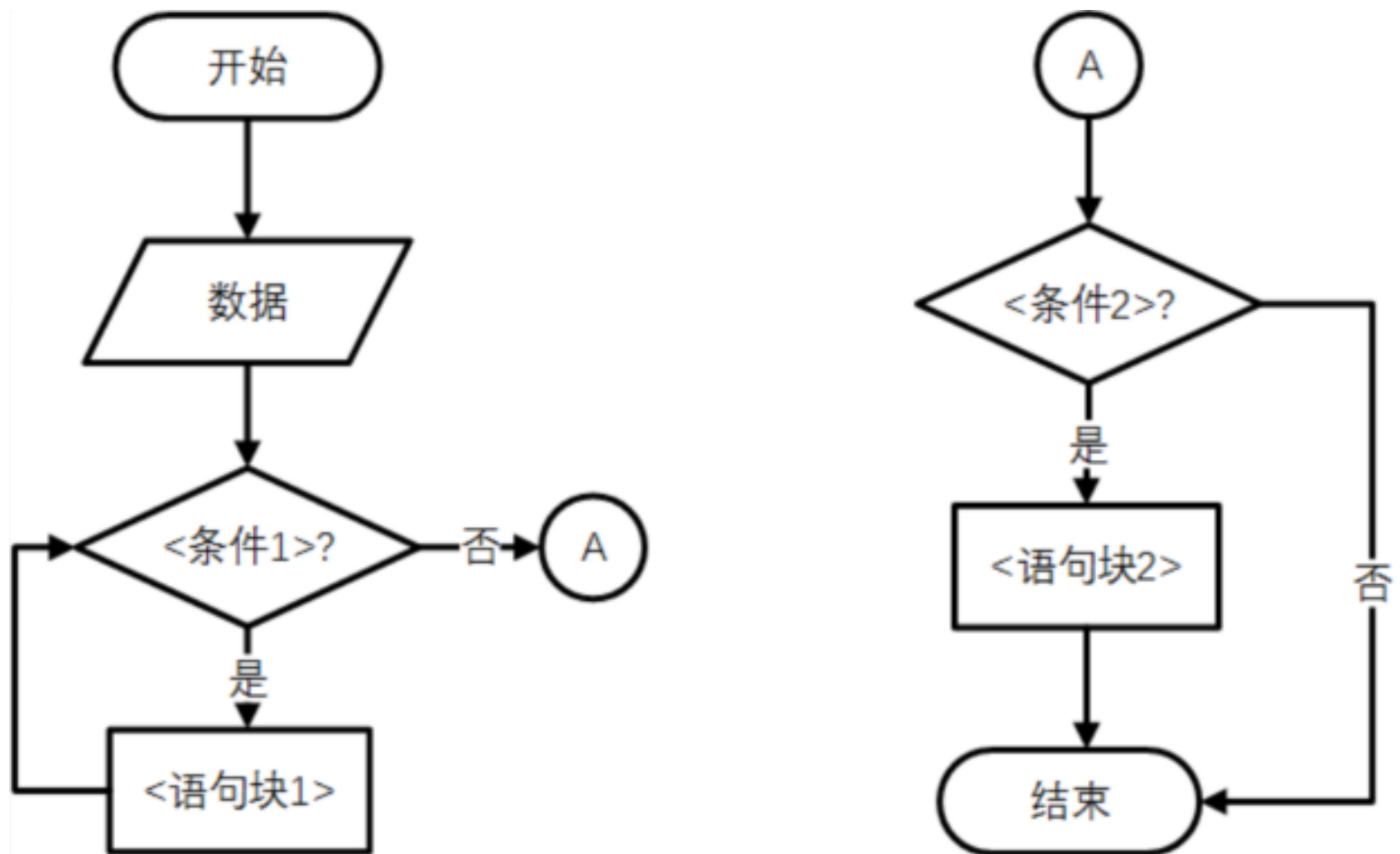


连接点



1.2 程序控制 程序的流程图

➤ 2. 程序的流程图





1.3 程序控制 程序的基本结构

- 顺序结构是程序的基础，但单一的顺序结构不可能解决所有问题。
- 程序由三种基本结构组成：

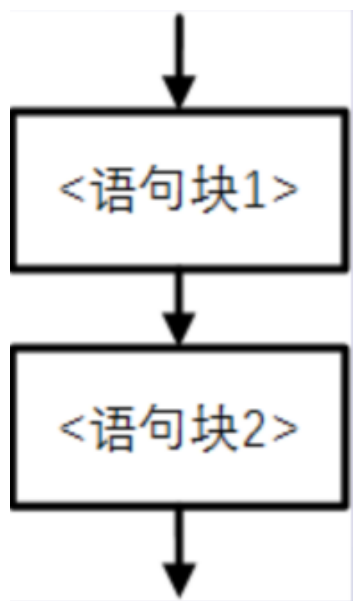
顺序结构 、 分支结构 、 循环结构

- 这些基本结构都有一个入口和一个出口。任何程序都由这三种基本结构组合而成



1.3 程序控制 程序的基本结构

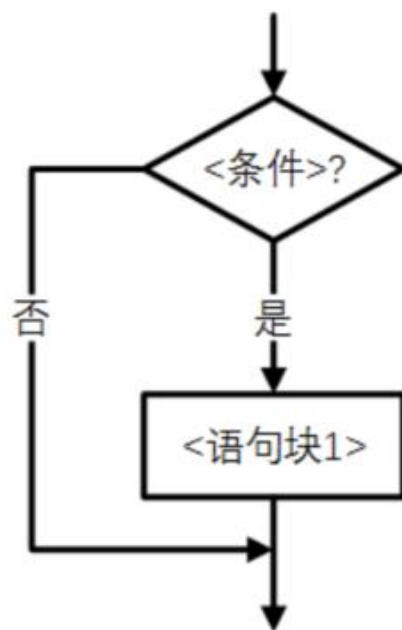
- 顺序结构是程序按照线性顺序依次执行的一种运行方式
- 语句块1和语句块2表示一个或一组顺序执行的语句



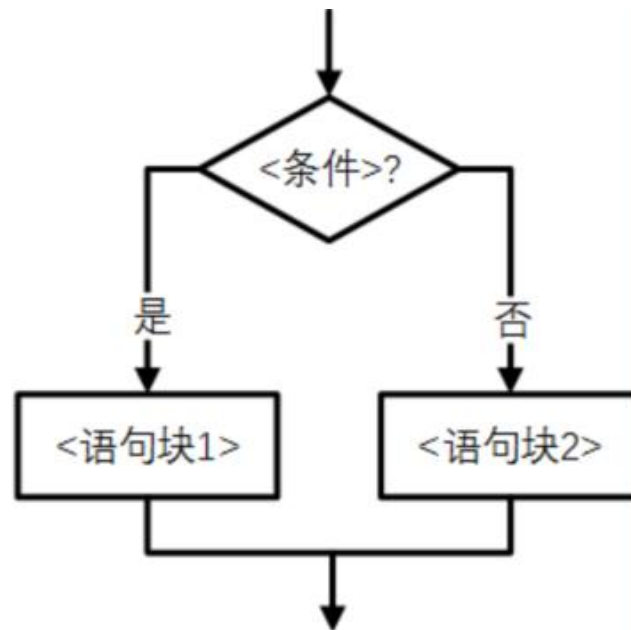
```
a = int(input())  
b = int(input())  
print(a + b)  
print(a - b)  
print(a * b)  
print(a / b)
```

1.3 程序控制 程序的基本结构

- 分支结构是程序根据条件判断结果而选择不同向前执行路径的一种运行方式
- 包括单分支结构和二分支结构。由二分支结构会组合形成多分支结构



单分支结构

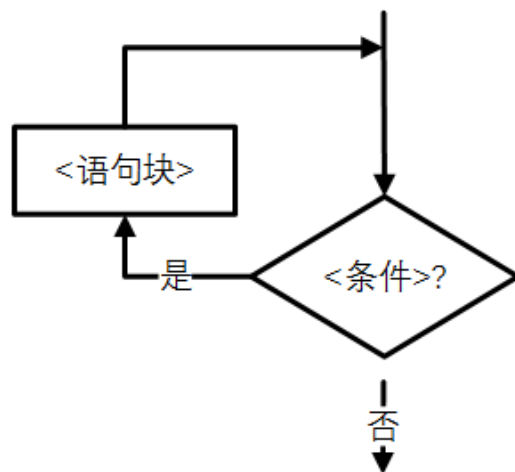


二分支结构

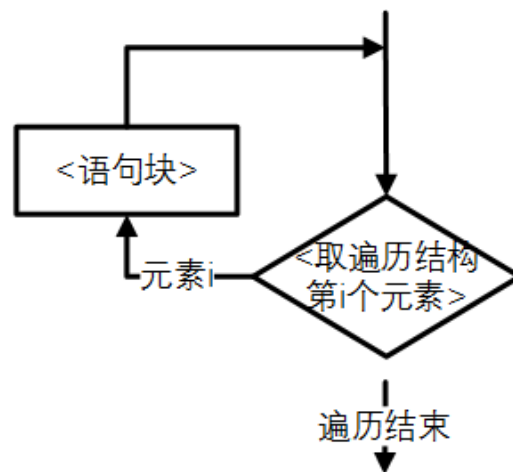


1.3 程序控制 程序的基本结构

- 循环结构是程序根据条件判断结果向后反复执行的一种运行方式，根据循环体触发条件不同，包括条件循环和遍历循环结构



条件循环



遍历循环

1.3 程序控制 程序的基本结构

- 对于一个计算问题，可以用IPO描述、流程图描述或者直接以Python代码方式描述

下面以累加计算为例进行说明

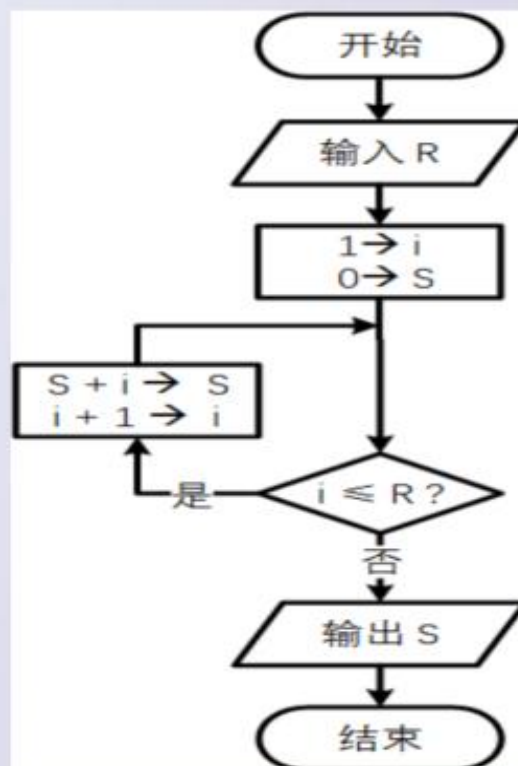
输入：正整数R

处理：

$$S=1+2+3+\cdots+R$$

输出：输出S

(a) 问题IPO描述



(b) 流程图描述

```
1 R = eval(input("请输入正整数:"))
2 i, S = 0, 0
3 while (i<=R):
4     S = S + i
5     i = i + 1
6 print("累加求和",S)
```

(c) Python代码描述



1.4 程序控制 条件结构

- 用到三个保留字：if/elif/else，if必须出现在条件结构的第一句，只有if和elif后面可以跟条件，满足则进入，不满足则不进入

```
# 判断闰年
year = int(input())
if year % 4 == 0 and year % 100 == 0:
    if year % 400 == 0:
        print("Yes")
    else:
        print("No")
elif year % 4 == 0 and year % 100 != 0:
    print("Yes")
else:
    print("No")
```

注意缩进：if语句后面跟着的代码块必须比上一个if语句多四个字符的缩进(4个空格或1个Tab)，缩进不正确会报错！

1.4 程序控制 条件结构



下面程序的运行结果是？

```
a = 100
if a % 3 == 0:
    print(1)
elif a % 2 == 0:
    print(2)
elif a % 10 == 0:
    print(3)
else:
    print(4)
```

2

进程已结束,退出代码0

在每一个条件结构中，最多进入1个代码块



1.5.1 程序控制 分支结构之单分支结构：if语句

- if语句的语法格式如下：

if <条件>:

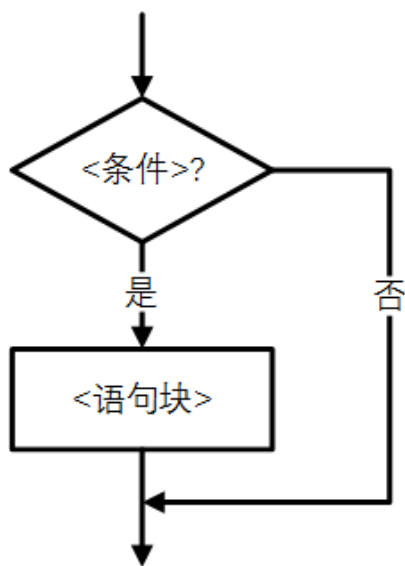
语句块

- 语句块是if条件满足后执行的一个或多个语句序列
- 语句块中语句通过与if所在行形成缩进表达包含关系
- if语句首先评估<条件>的结果值，如果结果为True，则执行语句块里的语句序列，然后控制转向程序的下一条语句。如果结果为False，语句块里的语句会被跳过



1.5.1 程序控制 分支结构之单分支结构：if语句

- If语句中语句块执行与否依赖于条件判断。
- 无论什么情况，控制都会转到if语句后与该语句同级别的下一条语句



```
PM = eval(input("请输入PM2.5数值: "))
if 0<= PM < 35:
    print("空气优质，快去户外运动!")
if 35 <= PM <75:
    print("空气良好，适度户外活动!")
if 75 <= PM:
    print("空气污染，请小心!")
```



1.5.2 程序控制 分支结构之单分支结构：if-else 语句

- 二分支结构，语法格式如下：

if <条件>:

 <语句块1>

else:

 <语句块2>

- <语句块1>是在if条件满足后执行的一个或多个语句序列
- <语句块2>是if条件不满足后执行的语句序列
- 二分支语句用于区分<条件>的两种可能True或者False，分别形成执行路径

```
PM = eval(input("请输入PM2.5 数值:"))
```

```
if PM >= 75:
```

```
    print("空气存在污染，请小心！")
```

```
else:
```

```
    print("空气没有污染，可以开展户外运动!")
```



1.5.2 程序控制 分支结构之单分支结构：if-else语句

- 适合通过判断返回特定值，语法格式如下：

〈表达式1〉 if 〈条件〉 else 〈表达式2〉

```
PM = eval(input("请输入PM2.5数值: "))  
print("空气{}污染!".format("存在" if PM >= 75 else "没有"))
```

- if…else的紧凑结构非常适合对特殊值处理的情况，如下

```
>>>count = 2
```

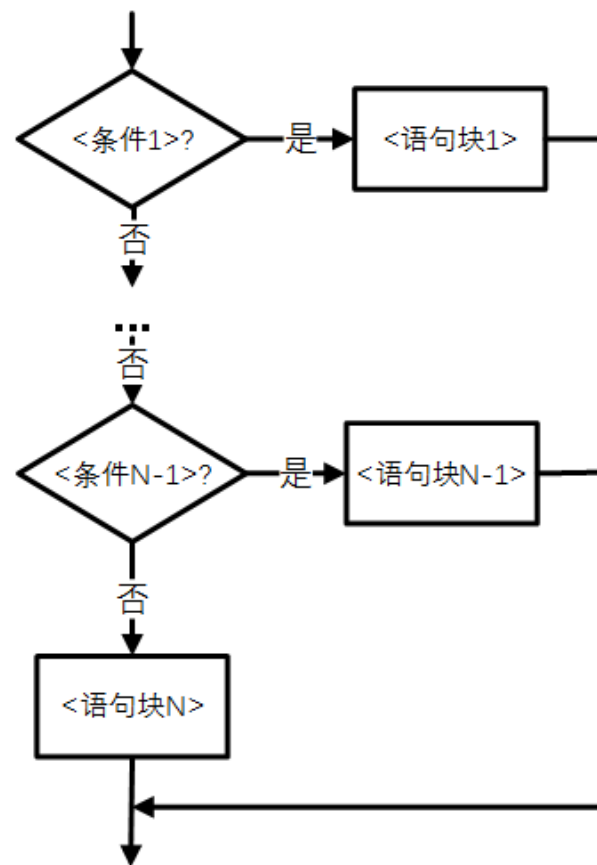
```
>>>count if count!=0 else "不存在"
```



1.5.3 程序控制 分支结构之单分支结构：if-elif-else语句

多分支结构，语句格式如下：

```
if <条件1>:  
    <语句块1>  
elif <条件2>:  
    <语句块2>  
...  
else:  
    <语句块N>
```





1.5.3 程序控制 分支结构之单分支结构: if-elif-else语句

- 多分支结构是二分支结构的扩展，这种形式通常用于设置同一个判断条件的多条执行路径。
- Python依次评估寻找第一个结果为True的条件，执行该条件下的语句块，同时结束后跳过整个if-elif-else结构，执行后面的语句。如果没有任何条件成立，else下面的语句块被执行。

```
PM = eval(input("请输入PM2.5数值: "))
if 0<= PM < 35:
    print("空气优质，快去户外运动!")
elif 35 <= PM <75:
    print("空气良好，适度户外活动！")
else:
    print("空气污染，请小心！")
```



1.5.4 程序控制 分支结构之课内练习

- 编一个根据体重和身高计算BMI值的程序，同时输出国际和国内的BMI指标建议值

BMI的定义如下：

$$\text{BMI} = \text{体重 (kg)} \div \text{身高}^2 (\text{m}^2)$$

分类	国际BMI值 (kg/m ²)	国内BMI值 (kg/m ²)
偏瘦	< 18.5	< 18.5
正常	18.5 ~ 25	18.5 ~ 24
偏胖	25 ~ 30	24 ~ 28
肥胖	>= 30	>= 28



1.5.4 程序控制 分支结构之课内练习

```
height, weight = eval(input("请输入身高(米)和体重(公斤)[逗号隔开]: "))
bmi = weight / pow(height, 2)
print("BMI 数值为: {:.2f}".format(bmi))
wto, dom = "", ""
if bmi < 18.5: # WHO 标准
    wto = "偏瘦"
elif bmi < 25: # 18.5 <= bmi < 25
    wto = "正常"
elif bmi < 30: # 25 <= bmi < 30
    wto = "偏胖"
else:
    wto = "肥胖"
if bmi < 18.5: # 我国卫生部标准
    dom = "偏瘦"
elif bmi < 24: # 18.5 <= bmi < 24
    dom = "正常"
elif bmi < 28: # 24 <= bmi < 28
    dom = "偏胖"
else:
    dom = "肥胖"
print("BMI 指标为:国际'{0}', 国内'{1}'".format(wto, dom))
```

请输入身高(米)和体重(公斤)[逗号隔开]:

1.65,84

BMI 数值为: 30.85

BMI 指标为:国际'肥胖', 国内'肥胖'

5.4 循环结构

- `range(a, b, step)`: 一个可迭代对象，目前可以先理解为一个list，生成一个以a开头，以step为步长的list，直到b为止(但不包括b)，step不写默认为1，a不写默认为0

`range(1, 4)` `[1, 2, 3]`

`range(2, 10, 2)` `[2, 4, 6, 8]`

`range(3)` `[0, 1, 2]`

`range(10, 0, -1)` `[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]`

5.4.1 程序的程序的循环结构 遍历循环：for语句

- 根据循环执行次数的确定性，循环可以分为确定次数循环和非确定次数循环
- 确定次数循环指循环体对循环次数有明确的定义
- 循环次数采用遍历结构中元素个数来体现
- Python通过保留字for实现“遍历循环”：

**for <循环变量> in <遍历结构>:
<语句块>**

5.4 程序的程序的循环结构 遍历循环：for语句

➤ 遍历结构可以是字符串、文件、组合数据类型或range()函数

循环N次

```
for i in range(N):  
    <语句块>
```

遍历文件fi的每一行

```
for line in fi:  
    <语句块>
```

遍历字符串s

```
for c in s:  
    <语句块>
```

遍历列表ls

```
for item in ls:  
    <语句块>
```



2. 循环结构

2.1.1 循环结构 遍历循环 for语句



- 遍历循环还有一种扩展模式，使用方法如下：

```
for <循环变量> in <遍历结构>:
```

```
<语句块1>
```

```
else:
```

```
<语句块2>
```

```
for s in "BIT":  
    print("循环进行中: " + s)  
else:  
    s = "循环正常结束"  
    print(s)
```

- 当for循环正常执行之后，程序会继续执行else语句中内容。
- else语句只在循环正常执行之后才执行并结束
- 可以在<语句块2>中放置判断循环执行情况的语句

2.1.2 循环结构 遍历循环: for...in...语句



for...in...循环：从一个list中依次取其元素直至取完

```
>>> List = [1, 2, 3, 4, 5, 7, 10, 22]
>>> for i in List:
    print(i)
```

```
1
2
3
4
5
7
10
22
```

2.2.1 循环结构 无限循环: while语句



无限循环一直保持循环操作直到特定循环条件不被满足才结束，不需要提前知道确定循环次数。

Python通过保留字while实现无限循环

while <条件>:
<语句块>

无限循环也有一种使用保留字else的扩展模式

while <条件>:
<语句块1>
else:
<语句块2>

2.2.1 循环结构 无限循环: while语句



while循环：满足while后的条件则进入循环，否则退出循环

```
# 计算从1到100的和。  
Sum = 0  
i = 1  
while i <= 100:  
    Sum = Sum + i  
    i = i + 1  
print(Sum)
```

2.2.1 循环结构 循环保留字



- 循环结构有两个辅助保留字： break和continue，用来辅助控制循环执行
- break用来跳出最内层for或while循环，脱离该循环后程序从循环后代码继续执行
- break语句跳出了最内层for循环，但仍然继续执行外层循环。每个break语句只有能力跳出当前层次循环

```
for s in "XMU":  
    for i in range(9):  
        print(s, end="")  
        if s=="M":  
            break
```

XXXXXXXXXXMUUUUUUUUUUU

2.2.1 循环结构 循环保留字



- continue用来结束当前当次循环，即跳出循环体中下面尚未执行的语句，但不跳出当前循环。
- 对于while循环，继续求解循环条件。而对于for循环，程序流程接着遍历循环列表。
- break语句则是结束整个循环过程，不再判断执行循环的条件是否成立

```
for s in "PYTHON":  
    if s=="T":  
        continue  
    print(s, end="")
```

PYHON

```
for s in "PYTHON":  
    if s=="T":  
        break  
    print(s, end="")
```

PY

2.2.1 循环结构 循环保留字



➤ break与continue:

break: 结束循环, continue: 跳过本次循环

```
# 列举1-100内的所有偶数
i = 1
while True:
    if i > 100:
        break
    i = i + 1
    if i % 2 != 0:
        continue
    print(i)
```

①当i大于100时, 退出循环,

break之后的语句都不执行。

②当i为奇数时, 跳过本次循环,

continue之后的语句都不执行。

③输出i

2.2.1 循环结构 循环保留字



- for循环和while循环中都存在一个else扩展用法
- else中的语句块只在一种条件下执行，即for循环正常遍历了所有内容没有因为break或return而退出。
- continue保留字对else没有影响

```
for s in "PYTHON":  
    if s=="T":  
        continue  
    print(s, end="")  
else:  
    print("正常退出")
```

```
for s in "PYTHON":  
    if s=="T":  
        break  
    print(s, end="")  
else:  
    print("正常退出")
```



3. Python杂项



3.1 Python杂项 Random库

- Python内置的random库主要用于产生各种分布的伪随机数序列。
- random库生成伪随机数序列，可用于除随机性要求高的加解密算法外的

大多数工程应用

使用random库主要目的是生成随机数

库提供了不同类型的随机数函数

所有函数都是基于random.random()函数扩展而来。

函数	描述
seed(a=None)	初始化随机数种子，默认值为当前系统时间
random()	生成一个[0.0, 1.0)之间的随机小数
randint(a, b)	生成一个[a,b]之间的整数
getrandbits(k)	生成一个k比特长度的随机整数
randrange(start, stop[, step])	生成一个[start, stop)之间以step为步数的随机整数
uniform(a, b)	生成一个[a, b]之间的随机小数
choice(seq)	从序列类型(例如：列表)中随机返回一个元素
shuffle(seq)	将序列类型中元素随机排列，返回打乱后的序列
sample(pop, k)	从pop类型中随机选取k个元素，以列表类型返回

3.2 Python杂项 模块的导入



- Python 模块(Module)，是一个 Python 文件，以 .py 结尾
- 包含了 Python 对象定义和Python语句
- 模块让你能够有逻辑地组织你的 Python 代码段。
- 模块把相关的代码分配到一个模块里能让你的代码更好用，更易懂
- 模块能定义函数，类和变量，模块里也能包含可执行的代码。



3.2 Python杂项 模块的导入

- 使用 `import` 语句来引入模块

```
import module1[, module2[,... moduleN]]
```

- 当解释器遇到 `import` 语句，如果模块在当前的搜索路径就会被导入。
- 搜索路径是一个解释器会先进行搜索的所有目录的列表。如想要导入模块 `support.py`，需要把命令放在脚本的顶端
- 一个模块只会被导入一次，不管你执行了多少次`import`。这样可以防止导入模块被一遍又一遍地执行



3.2 Python杂项 模块的导入 from...import 语句

- Python 的 `from` 语句让你从模块中导入一个指定的部分到当前命名空间中

```
from modname import name1[, name2[, ... nameN]]
```

例：要导入模块 `fib` 的 `fibonacci` 函数，使用如下语句

```
from fib import fibonacci
```

这个声明不会把整个 `fib` 模块导入到当前的命名空间中，它只会将 `fib` 里的 `fibonacci` 单个引入到执行这个声明的模块的全局符号表



3.2 Python杂项 模块的导入：路径和Python变量

➤ 当你导入一个模块，Python 解析器对模块位置的搜索顺序是：

1、当前目录

2、如果不在当前目录，Python 则搜索在 shell 变量PYTHONPATH 下的每个目录。

3、如果都找不到，Python会察看默认路径。

Windows下：模块搜索路径存储在 system 模块的 sys.path 变量中。变量里包含当前目录，PYTHONPATH和由安装过程决定的默认目录。

UNIX下：默认路径一般为/usr/local/lib/python/。



3.2 Python杂项 模块的导入：路径和Python变量

- 作为环境变量，PYTHONPATH 由装在一个列表里的许多目录组成。PYTHONPATH 的语法和 shell 变量 PATH 的一样。
- 在 Windows 系统，典型的 PYTHONPATH 如下：

```
set PYTHONPATH=c:\python27\lib;
```

- 在 UNIX 系统，典型的 PYTHONPATH 如下：

```
set PYTHONPATH=/usr/local/lib/python
```



4. 实践

- 蒙特卡罗 (Monte Carlo) 方法, 又称随机抽样或统计试验方法。当所要求解的问题是某种事件出现的概率, 或者是某个随机变量的期望值时, 它们可以通过某种“试验”的方法, 得到这种事件出现的频率, 或者这个随机变数的平均值, 并用它们作为问题的解。这就是蒙特卡罗方法的基本思想。

应用蒙特卡罗方法求解 π 的基本步骤如下:

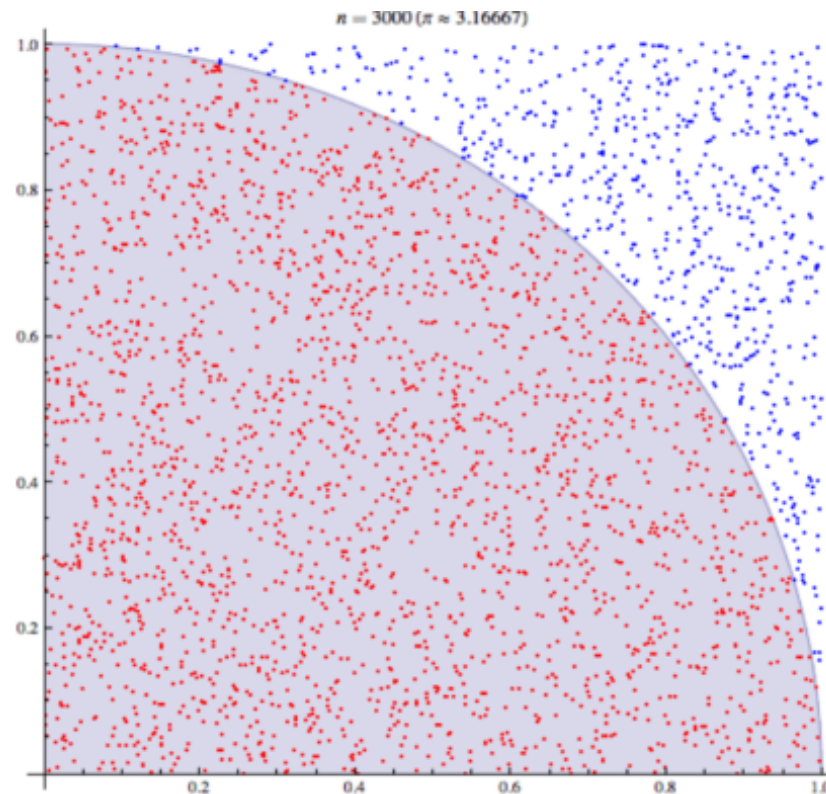
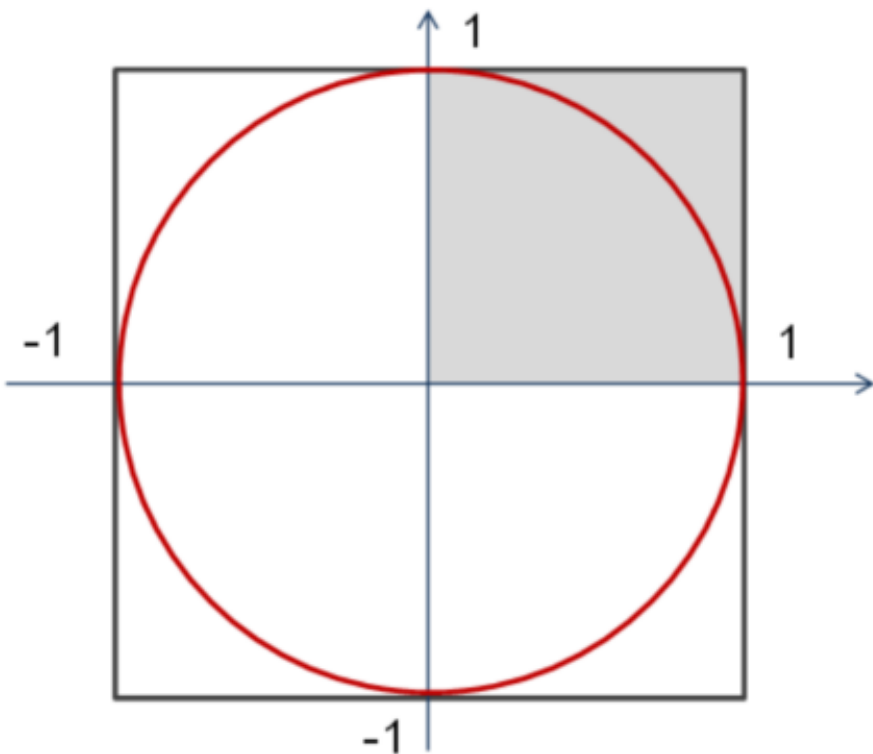
- 随机向单位正方形和圆结构, 抛洒大量“飞镖”点
- 计算每个点到圆心的距离从而判断该点在圆内或者圆外
- 用圆内的点数除以总点数就是 $\pi/4$ 值。

4.实践 π 的计算



厦门大学
XIAMEN UNIVERSITY

程序控制例子： π 的计算



随机点数量越大，越充分覆盖整个图形，计算得到的 π 值越精确。实际上，这个方法的思想是利用离散点值表示图形的面积，通过面积比例来求解 π 值。

4.实践 π 的计算



```
from random import random
from math import sqrt
from time import process_time
DARTS = 10000
hits = 0.0
process_time()
for i in range(1, DARTS+1):
    x, y = random(), random()
    dist = sqrt(x ** 2 + y ** 2)
    if dist <= 1.0:
        hits = hits + 1
pi = 4 * (hits/DARTS)
print("Pi值是{}".format(pi))
print("运行时间是: {:.5}s".format(process_time()))
```

DARTS	π	运行时间
2^{10}	3. 109375	0. 011s
2^{11}	3. 138671	0. 012s
2^{12}	3. 150390	0. 014s
2^{13}	3. 143554	0. 018s
2^{14}	3. 141357	0. 030s
2^{15}	3. 147827	0. 049s
2^{16}	3. 141967	0. 116s
2^{18}	3. 144577	0. 363s
2^{20}	3. 1426696777	1. 255s
2^{25}	3. 1416978836	40. 13s

4.实践 练习



厦门大学
XIAMEN UNIVERSITY

- 1. 完成单分支列表PM2.5结果判断程序
- 2. 完成单分支列表PM2.5结果判断程序。
- 3. 完成BMI计算程序练习。
- 4. 完成 π 的计算程序。
- 5. 完成下列程序。

4.实践 练习



厦门大学
XIAMEN UNIVERSITY

本题要求统计一个整型序列中出现次数最多的整数及其出现次数。

输入格式:

输入在一行中给出序列中整数个数 N ($0 < N \leq 1000$)，以及 N 个整数。数字间以空格分隔。

输出格式:

在一行中输出出现次数最多的整数及其出现次数，数字间以空格分隔。题目保证这样的数字是唯一的。

输入样例:

```
10 3 2 -1 5 3 4 3 0 3 2
```

输出样例:

```
3 4
```

```
List = list(map(int, input().split()))
Array = List[1:]
Set = set(Array)
MaxCount = -1
MaxNumber = 0
for i in Set:
    if Array.count(i) > MaxCount:
        MaxCount = Array.count(i)
        MaxNumber = i
print(MaxNumber, MaxCount)
```

4.实践 练习



厦门大学
XIAMEN UNIVERSITY

一群猴子要选新猴王。新猴王的选择方法是：让N只候选猴子围成一圈，从某位置起顺序编号为1~N号。从第1号开始报数，每轮从1报到3，凡报到3的猴子即退出圈子，接着又从紧邻的下一只猴子开始同样的报数。如此不断循环，最后剩下的一只猴子就选为猴王。请问是原来第几号猴子当选猴王？

输入格式：

输入在一行中给一个正整数N (≤ 1000) 。

输出格式：

在一行中输出当选猴王的编号。

输入样例：

11

输出样例：

7

```
1  N = int(input())
2  List = []
3  for i in range(1, N + 1):
4      List.append(i)
5  Count = 1
6  while len(List) > 1:
7      for i in List:
8          if Count == 3:
9              if i == List[len(List) - 1]:
10                 Count = 0
11             else:
12                 Count += 1
13             List.remove(i)
14             Count += 1
15  print(List[0])
```

```
import jieba
excludes = {"将军","却说","荆州","二人","不可","不  
能","如此"}
txt = open("三国演义.txt", "r", encoding='utf-  
8').read()
words = jieba.lcut(txt)
counts = {}
for word in words:
    if len(word) == 1:
        continue
    elif word == "诸葛亮" or word == "孔明曰":
        rword = "孔明"
    elif word == "关公" or word == "云长":
        rword = "关羽"
    elif word == "玄德" or word == "玄德曰":
        rword = "刘备"
    elif word == "孟德" or word == "丞相":
        rword = "曹操"
    else:
        rword = word
    counts[rword] = counts.get(rword,0) + 1
```

搜索词频

1. conda 安装jieba 模块

激活环境: conda activate xxxxx

安装jieba: conda install -c conda-forge jieba

2. 下载“三国演义.txt”到windosw C盘 用户目录下
(和jupyter notebook文件在同一个文件夹)

```
for word in excludes:
    del(counts[word])
items = list(counts.items())
items.sort(key=lambda x:x[1], reverse=True)
for i in range(5):
    word, count = items[i]
    print ("{0:<10}{1:>5}".format(word, count))
```

4.实践 练习



厦门大学
XIAMEN UNIVERSITY

一个矩阵元素的“鞍点”是指该位置上的元素值在该行上最大、在该列上最小。

本题要求编写程序，求一个给定的 n 阶方阵的鞍点。

输入格式：

输入第一行给出一个正整数 n ($1 \leq n \leq 6$)。随后 n 行，每行给出 n 个整数，其间以空格分隔。

输出格式：

输出在一行中按照“行下标 列下标”（下标从0开始）的格式输出鞍点的位置。如果鞍点不存在，则输出“NONE”。题目保证给出的矩阵至多存在一个鞍点。

输入样例1：

```
4
1 7 4 1
4 8 3 6
1 6 1 2
0 7 8 9
```

输出样例1：

```
2 1
```

```
1  N = int(input())
2  Matrix = []
3  for i in range(N):
4      List = list(map(int, input().split()))
5      Matrix.append(List)
6  for a in range(N):
7      for b in range(N):
8          if Matrix[a][b] == max(Matrix[a]):
9              Min = Matrix[0][b]
10             for c in range(1, N):
11                 if Matrix[c][b] < Min:
12                     Min = Matrix[c][b]
13             if Matrix[a][b] == Min:
14                 print(a, b)
15                 exit(0)
16  print("NONE")
```